

Progetto per Programmazione ad oggetti

CACTUS!

Chiara Mazzoni,
Sofia Molari,
Arianna Mondardini,
Rebecca Olivieri

30 Giugno 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	4
2	Design	6
2.1	Architettura	6
2.2	Design dettagliato	9
2.2.1	Mazzoni Chiara	9
2.2.2	Molari Sofia	11
2.2.3	Mondardini Arianna	15
2.2.4	Olivieri Rebecca	18
3	Sviluppo	22
3.1	Testing automatizzato	22
3.2	Note di sviluppo	20
4	Commenti finali	24
4.1	Autovalutazione e lavori futuri	24
4.1.1	Mazzoni Chiara	24
4.1.2	Molari Sofia	24
4.1.3	Mondardini Arianna	25
4.1.4	Olivieri Rebecca	26
A	Guida utente	27
A.1	Configurazione iniziale	27
A.2	Schermata di gioco	28
A.3	Scarto simultaneo	29
A.4	Specifica delle carte del gioco	30
B	Esercitazioni di laboratorio	33
B.0.1	chiara.mazzoni12@studio.unibo.it	33
B.0.2	sofia.molari@studio.unibo.it	33
B.0.3	arianna.mondardini@studio.unibo.it	33
B.0.4	rebecca.olivieri2@studio.unibo.it	34

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il gioco è un card-game singleplayer, ispirato all'omonimo gioco di carte "Cactus!", basato su memoria, strategia e rapidità di reazione. La partita si svolge tra quattro giocatori. L'obiettivo del giocatore è terminare la partita con il punteggio più basso possibile, minimizzando il valore complessivo delle carte presenti davanti a sé.

Il gioco si svolge con un mazzo da 40 carte. In "Cactus!", il valore delle carte determina il punteggio, con alcune eccezioni per le figure:

- Le carte numerate dall'Asso al 7 valgono quanto il loro numero.
- Il Re è la carta più ambita, poiché vale 0 punti.
- Il Fante vale 8 punti.
- Il Cavallo vale 9 punti.

Ogni giocatore inizia con 4 carte coperte, disposte in fila. La sfida risiede nell'incertezza: all'inizio della partita, il giocatore può visualizzare solo due delle sue quattro carte (una sola volta). Per il resto del gioco, dovrà ricordare la posizione e il valore di tali carte che rimarranno non visibili fino all'attivazione di specifiche azioni di gioco.

Durante il proprio turno, il giocatore pesca una carta dal mazzo coperto, la visualizza e decide se scambiarla con una delle proprie carte, altrimenti la carta verrà scartata. In quest'ultimo caso, se la carta possiede un Potere Speciale, il giocatore può scegliere di attivarlo.

Solo alcune carte possiedono un Potere Speciale:

- Il numero 6: Permette di guardare una propria carta coperta.
- Il numero 7: Permette di scambiare di posto due carte qualsiasi sul tavolo senza guardarle.
- Il Fante: Permette di rivelare una carta qualsiasi sul tavolo (propria o avversaria), che resterà scoperta fino al termine della partita.

Inoltre, il gioco prevede un'ulteriore regola detta "Scarto Simultaneo": se un giocatore scarta una carta, chiunque pensi di avere una carta di identico valore può tentare di scartarla per primo. Se l'azione è corretta, il giocatore si libera di una carta; se sbaglia,

riceve una carta di penalità dal mazzo. Se un giocatore finisce tutte le carte il gioco termina immediatamente. Il gioco può terminare anche quando un giocatore, ritenendo di avere il punteggio minore, dichiara "Cactus", concedendo un ultimo turno agli avversari prima del conteggio finale. A parità di punteggio vince il giocatore con il minor numero di carte.

Requisiti funzionali

- Il sistema deve distribuire 4 carte coperte a ciascun giocatore, prendendole dal mazzo di 40 carte.
- Il sistema deve permettere al giocatore di visualizzare 2 carte su 4 a inizio partita.
- Il sistema deve consentire al giocatore di pescare dal mazzo e scartare.
- Il sistema deve consentire al giocatore di scambiare la carta pescata con una di quelle del proprio mazzo.
- Il sistema deve consentire a ciascun giocatore lo "Scarto Simultaneo" con validazione del valore e conseguente penalizzazione se necessaria.
- Il sistema deve consentire al giocatore di attivare i "Poteri Speciali" delle carte al momento dello scarto.
- Il sistema deve gestire la chiamata "Cactus!" e l'ultimo giro di gioco, determinando il vincitore.

Requisiti non funzionali

- Data la meccanica dello scarto rapido, il sistema deve garantire tempi di risposta brevi, percepiti come immediati dall'utente.
- L'interfaccia grafica deve essere chiara e consentire al giocatore di comprendere in ogni momento lo stato della partita, il turno corrente, le azioni disponibili e i risultati delle azioni effettuate.
- Il sistema deve calcolare in modo corretto e deterministico il punteggio e l'assegnazione delle penalità per lo scarto errato.
- Il sistema deve garantire la riservatezza delle informazioni relative alle carte coperte, rendendole accessibili esclusivamente nei casi previsti dalle regole di gioco.
- Il sistema deve garantire il corretto funzionamento su diversi sistemi operativi senza modifiche al codice sorgente.

1.2 Modello del Dominio

Il dominio di CACTUS! ruota attorno a una partita (Game) che coinvolge quattro giocatori e si articola in una sequenza di turni. Ogni turno (Round) è associato a un giocatore corrente e definisce le azioni che questo può compiere.

Ciascun giocatore (Player) possiede una mano di quattro carte (PlayerHand): le carte sono inizialmente coperte e possono essere rivelate o scambiate solo attraverso azioni di gioco specifiche.

Il mazzo di pesca (DrawPile) è la sorgente da cui i giocatori attingono durante il turno; il mazzo degli scarti (DiscardPile) raccoglie le carte eliminate, la cui carta in cima è sempre visibile e innesca lo scarto simultaneo.

Una carta (Card) ha un valore numerico, un seme (Suit) e un punteggio, che non sempre coincide con il valore (il Re vale 0). Alcune carte possiedono un potere speciale (SpecialPower), attivabile al momento dello scarto: guardare una propria carta coperta (6), scambiare due carte senza guardarle (rivelare permanentemente una carta sul tavolo (7), rivelare permanentemente una carta sul tavolo (Fante).

La partita termina quando un giocatore chiama "Cactus!", dopo di che viene concesso un ultimo turno agli avversari prima del conteggio finale dei punteggi.

Un secondo aspetto critico è la meccanica dello scarto simultaneo: si tratta di un'azione che può essere tentata da qualsiasi giocatore fuori dal proprio turno, richiedendo quindi che il dominio modelli esplicitamente le azioni disponibili sia nel turno regolare sia in risposta a eventi esterni.

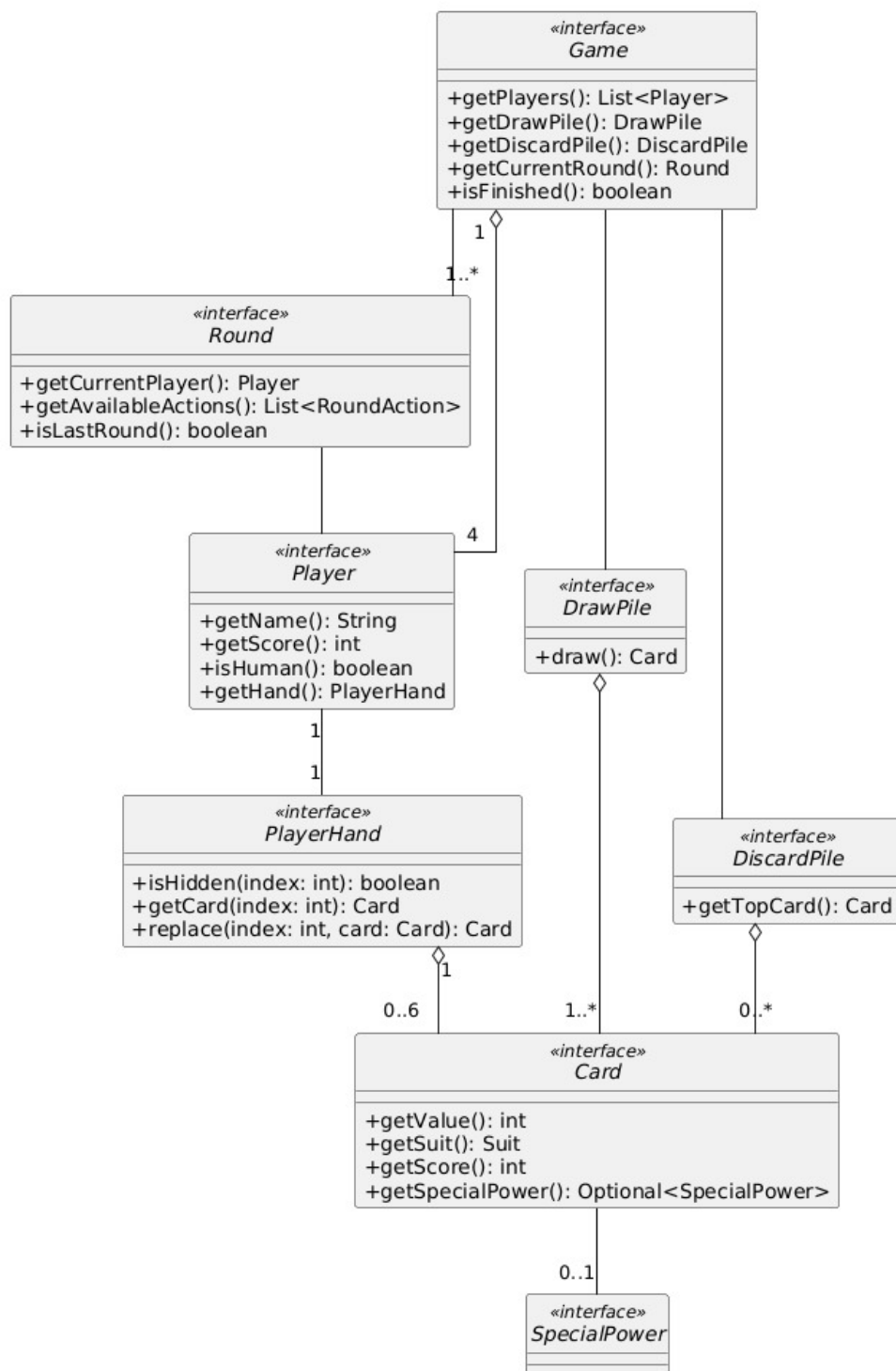


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro.

Capitolo 2

Design

2.1 Architettura

L'architettura di CACTUS! segue il pattern architetturale MVC. Il Model ha come punto d'ingresso l'interfaccia **Game**, che espone lo stato complessivo della partita: i giocatori, i mazzi e il turno corrente (**Round**). La View è rappresentata dall'interfaccia **GameView**, responsabile esclusivamente della presentazione dello stato e delle azioni disponibili. Il Controller, con punto d'ingresso l'interfaccia **Controller** (implementata da **ControllerImpl**), orchestra il flusso di gioco: riceve gli input dell'utente, li traduce in azioni sul Model e notifica la View di aggiornarsi. La comunicazione fra i componenti avviene su due canali distinti, entrambi orientati verso il Controller. Il primo segue il pattern Observer: il Controller implementa l'interfaccia **GameObserver**, attraverso la quale il Model notifica l'avanzamento dei turni, le modifiche di stato e la fine della partita. Il secondo canale va dalla View al Controller: la View comunica gli input dell'utente invocando i metodi dell'interfaccia **GameViewListener**, anch'essa implementata dal Controller. In direzione opposta il Controller aggiorna attivamente la View invocando i metodi di **GameView** e passandole uno snapshot immutabile dello stato da visualizzare. Il Controller ricopre tre ruoli: fa avanzare la partita coordinando le tre fasi (configurazione, svolgimento dei turni e fine partita) delegando la decisione alla strategia del giocatore corrente quando questo non è umano e scandendo l'avanzamento temporizzato dei bot tramite il metodo `tick()`, invocato periodicamente da un timer esterno; reagisce all'input dell'utente traducendolo nelle corrispondenti azioni di dominio da applicare al Model; reagisce ai cambiamenti del Model propagandone gli effetti alla View. Poiché ciascun componente dipende soltanto da interfacce, e mai dalle implementazioni concrete degli altri, la sostituzione in blocco della View lascia inalterati gli altri due livelli.

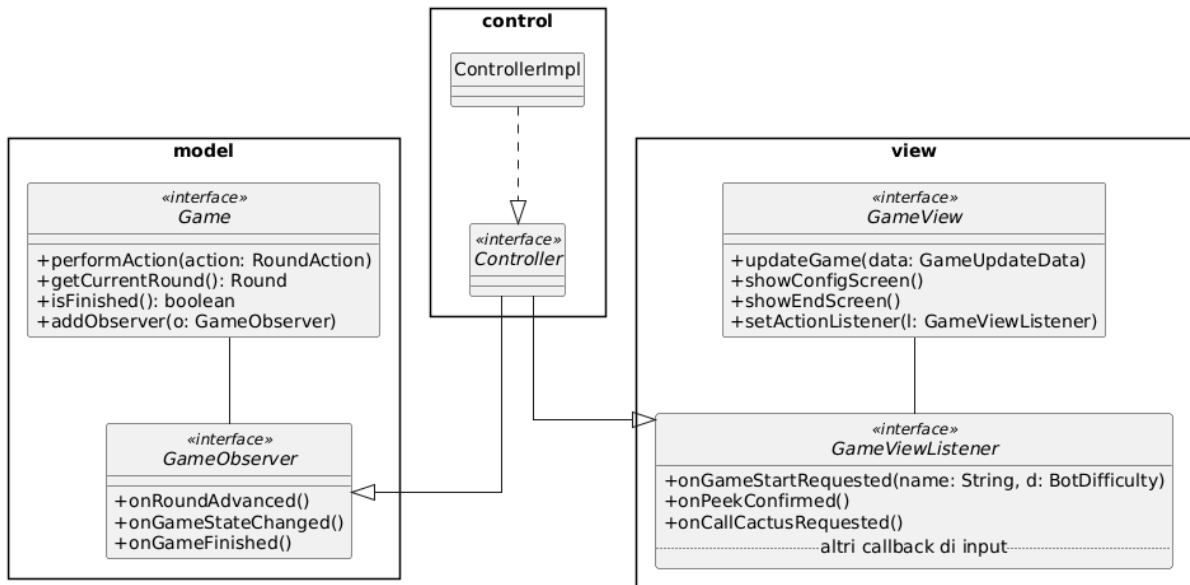


Figura 2.1: Schema UML architetturale di CACTUS!

2.2 Design dettagliato

2.2.1 Mazzoni Chiara

Gestione sicura dell'assenza di carte nelle pile

Problema. Le pile di pesca (`DrawPileImpl`) e di scarto (`DiscardPileImpl`) possono trovarsi in uno stato vuoto durante il gioco: il mazzo di pesca si esaurisce nel corso della partita, mentre la pila degli scarti è inizialmente vuota. È necessario gestire il caso in cui vengano fatte operazioni come `draw()` o `getTopCard()` su una pila vuota. Restituire `null` esporrebbe il codice a eventuali `NullPointerException`, mentre lanciare un'eccezione non sarebbe stata la scelta migliore, trattandosi di un evento normale e atteso durante il gioco.

Soluzione. Si è scelto di far restituire ai metodi `draw()` e `getTopCard()` un oggetto di tipo `Optional<Card>`. In questo modo l'assenza di una carta, e quindi il caso di mazzo vuoto, viene gestito in modo esplicito e il codice risulta più robusto. Inoltre si è preferito organizzare le carte usando come struttura dati l'interfaccia `Deque` (implementata tramite `ArrayDeque`); l'utilizzo dei suoi metodi `poll()` e `peek()` permette di estrarre le carte in sicurezza, evitando di mandare il gioco in crash.

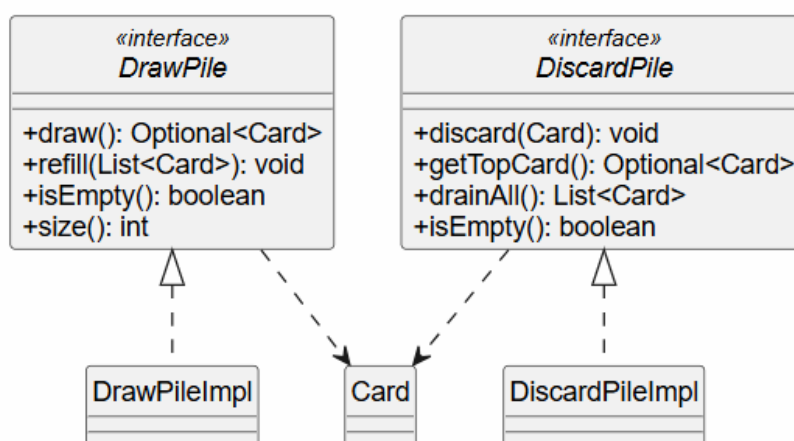


Figura 2.2: Rappresentazione UML dell'utilizzo di `Optional` nelle firme dei metodi

Separazione tra persistenza e calcolo delle statistiche

Problema. L'applicazione si deve occupare sia di salvare e caricare i risultati delle partite da un file, sia di calcolare statistiche complesse (come il numero di vittorie relative a ogni giocatore, la media dei round in cui si conclude una partita, la classifica generale dei giocatori e le relative partite vinte). Se il **Controller** avesse dovuto gestire direttamente il file JSON e i calcoli per le statistiche, la classe sarebbe diventata eccessivamente grande e complessa; inoltre, si sarebbe violato il *Single Responsibility Principle* (SRP).

Soluzione. Si è deciso di separare le responsabilità e gestire l'archiviazione dei dati del gioco applicando il *Facade Pattern*. È stata creata l'interfaccia `HistoryManager` (con la rispettiva implementazione `HistoryManagerImpl`), utilizzata come unico punto d'accesso da parte del **Controller** e che coordina due componenti indipendenti:

- **HistoryRepository**: interfaccia responsabile esclusivamente del caricamento e del salvataggio dei dati delle partite su file JSON (implementata da `JsonHistoryRepository` tramite la libreria `Gson`).
- **StatsCalculator**: classe responsabile esclusivamente del calcolo delle statistiche a partire dai dati caricati.

In questo modo, `HistoryManager` espone al `Controller` un'interfaccia semplice con soli due metodi (`save()` e `getStats()`), nascondendo la complessità interna e l'eterogeneità delle operazioni.

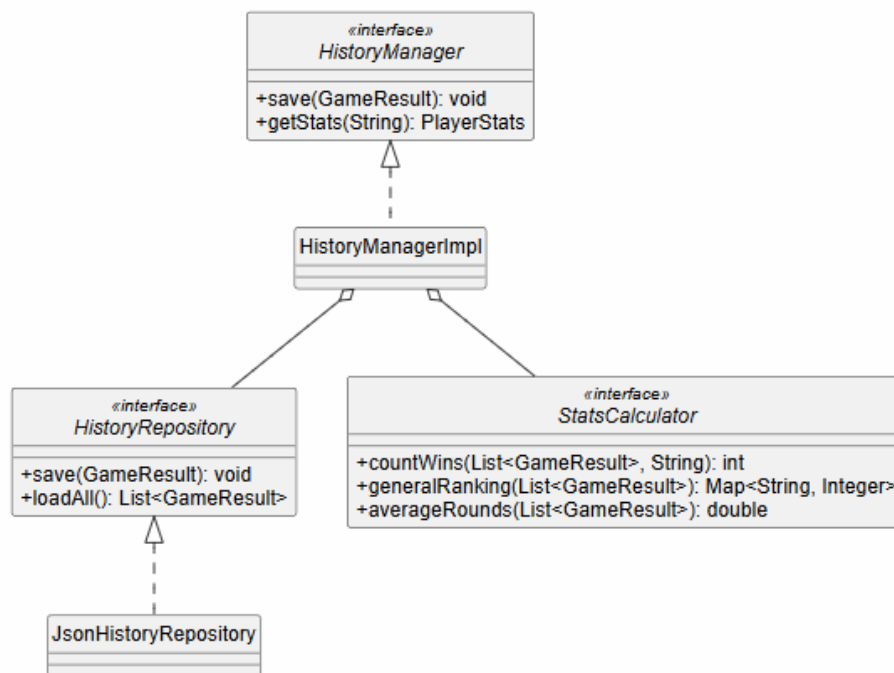


Figura 2.3: Rappresentazione UML dell'interfaccia `HistoryManager` che utilizza, tramite `HistoryManagerImpl`, le interfacce `HistoryRepository` e `StatsCalculator`

Rappresentazione immutabile dei dati di gioco

Problema I risultati di una partita completata e le statistiche di un giocatore sono dati che, una volta prodotti dal `Model`, devono essere passati a `View`, `Controller` e file JSON, ed è importante che risulti impossibile modificarli. Rappresentarli con classi ordinarie richiederebbe la scrittura manuale di parti di codice come costruttori, `getter` e `toString`, il che aumenterebbe il rischio di introdurre bug e aggiungerebbe complessità all'implementazione.

Soluzione Si è scelto di implementare `GameResult` e `PlayerStats` utilizzando i `Record`. Questa scelta è dovuta ad alcune caratteristiche rilevanti di questo costrutto:

- È immutabile, il che risulta ideale per fornire a `View` e `Controller` un formato di sola lettura del risultato, evitando modifiche non consentite e garantendo sicurezza nel passaggio dei dati.

- I metodi standard vengono generati in modo automatico, il che consente di avere un codice chiaro e pulito. Questo esplicita anche il fatto che la classe sia solo un portatore di dati.
- È una feature moderna del linguaggio Java e rispecchia la natura dei dati rappresentati: un risultato di partita o uno snapshot di statistiche non ha senso che cambi dopo la sua creazione.

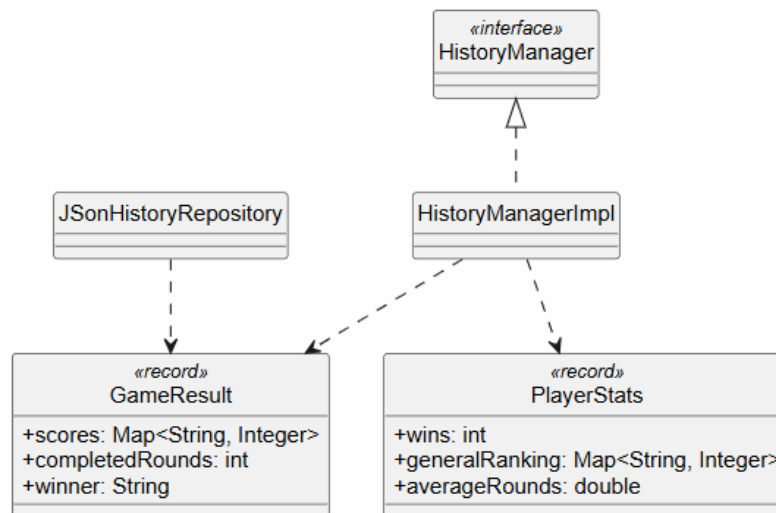


Figura 2.4: Rappresentazione UML dell'utilizzo di record per la rappresentazione immutabile dei dati di gioco

2.2.2 Molari Sofia

Le strategie dei bot

Problema. Il gioco prevede tre avversari controllati dal calcolatore, con quattro livelli di difficoltà (EASY, MEDIUM, HARD, VERY_HARD). Ogni livello deve poter scegliere le proprie azioni con una logica diversa, e si vuole poter aggiungere o cambiare un livello senza che il giocatore bot debba conoscere i dettagli di ciascuna difficoltà.

Soluzione. Si è applicato il pattern Strategy. La scelta dell'azione è astratta nell'interfaccia `BotStrategy`, di cui `BotPlayer` è il Context: `BotPlayer` non conosce la difficoltà che sta usando, ma delega `chooseAction(round)` e `performInitialPeek(hand)` alla strategia ricevuta nel costruttore. Le strategie concrete sono `EasyBotStrategy`, `MediumBotStrategy`, `HardBotStrategy` e `VeryHardBotStrategy`, e si distinguono per quanta informazione sfruttano: `Easy` sceglie a caso fra le azioni disponibili e ignora i poteri, `Medium` decide guardando solo le carte visibili sul tavolo, `Hard` sfrutta la memoria delle carte spiate per guidare scambi e scarti, `VeryHard` estende `Hard` ottimizzando l'uso dei poteri speciali e chiamando "Cactus!" solo quando stima di essere in vantaggio su tutti. Aggiungere una nuova difficoltà richiede solo una nuova implementazione di `BotStrategy`, registrata nella factory `BotStrategyFactory`, senza alcuna modifica a `BotPlayer`.

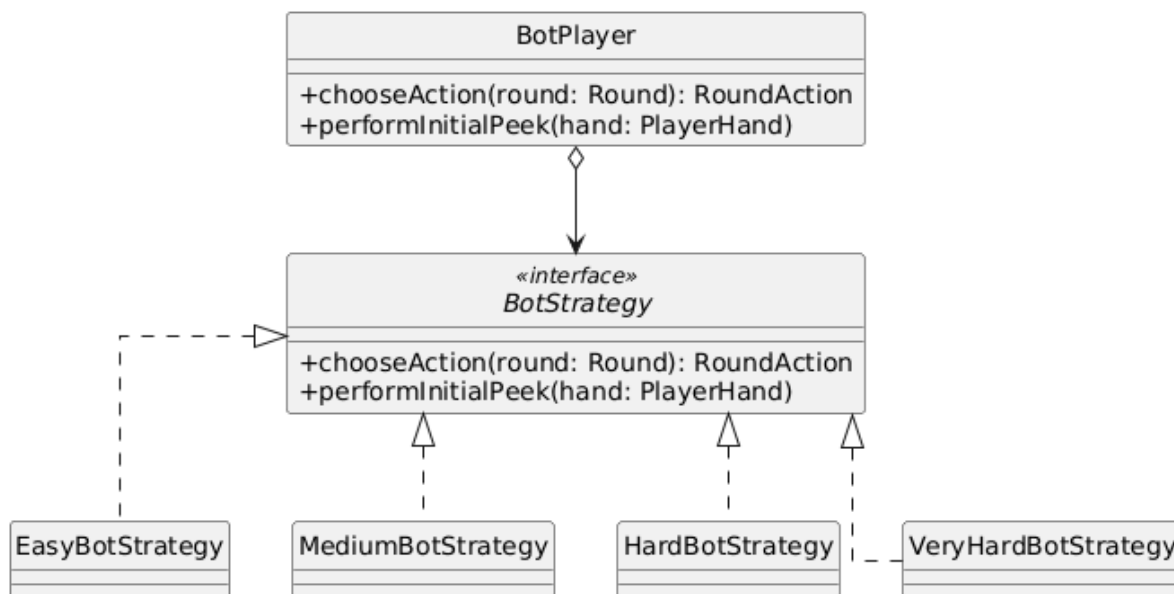


Figura 2.5: Rappresentazione UML del pattern Strategy: BotStrategy è la strategia, BotPlayer è il Context, le quattro implementazioni concrete sono le strategie intercambiabili.

Riuso del comportamento delle strategie

Problema. Le quattro strategie condividono gran parte del comportamento, in particolare il modo in cui un turno viene gestito fase per fase; le due strategie con memoria (Hard e VeryHard) condividono inoltre tutta la logica basata sulle carte spiate. Implementare ogni strategia in modo indipendente avrebbe portato a duplicare questo comportamento comune.

Soluzione. Si è applicato il pattern Template Method. La classe astratta AbstractBotStrategy definisce il metodo template chooseAction, dichiarato final, che fissa lo scheletro della decisione: in base alla fase corrente del turno gestisce direttamente la sola fase DRAW e delega le altre quattro a metodi astratti specializzati (chooseDecision, chooseSpecialPower, chooseEndTurn, chooseSimultaneousDiscard), che le sottoclassi riempiono senza poter alterare l'ordine delle fasi. Una seconda classe astratta, AbstractMemoryBotStrategy, si inserisce fra il livello base e le strategie con memoria: implementa quei metodi appoggiandosi a una BotMemory e delega a sua volta i pochi punti che ancora variano, tramite un nuovo metodo astratto (handleRevealPower) e un metodo opzionale con implementazione di default (handleSwapPowerFallback). In questo modo HardBotStrategy e VeryHardBotStrategy ridefiniscono solo questi due punti ed ereditano l'intera logica con memoria, e le parti variabili dell'algoritmo risultano riempite a due livelli diversi della gerarchia. La memoria è iniettata dalla factory BotStrategyFactory, che la fornisce alle sole strategie che ne hanno bisogno, mantenendo separata la gestione dello stato dalla logica di decisione. L'alternativa di tenere quattro strategie indipendenti senza gerarchia avrebbe duplicato la logica con memoria fra Hard e VeryHard, motivo per cui si è scelto di fattorizzare il comportamento comune nelle due classi astratte.

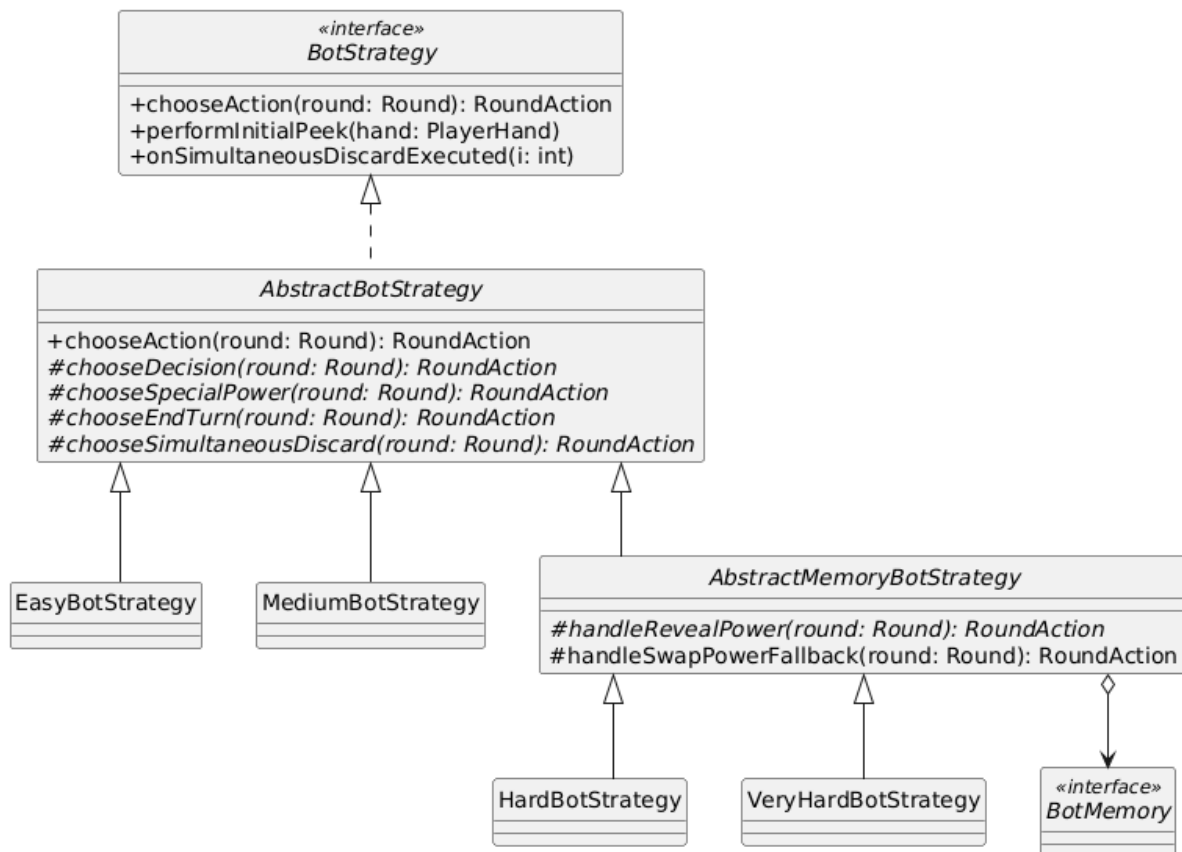


Figura 2.6: Rappresentazione UML del pattern Template Method applicato alla gerarchia delle strategie: `chooseAction` è il metodo template e i passi variabili sono riempiti a due livelli (`AbstractBotStrategy` e `AbstractMemoryBotStrategy`).

Comunicazione tra View e Controller

Problema. La View deve notificare al Controller gli input dell'utente (avvio della partita, conferma del peek iniziale, attivazione di un potere, scarto, chiamata di "Cactus!"). Se la View dipendesse dalla classe concreta del Controller si creerebbe un accoppiamento rigido, che renderebbe difficile sostituire l'una o l'altro. Inoltre View e Controller si riferenziano a vicenda, dato che il Controller aggiorna la View e la View notifica il Controller: questo genera una dipendenza circolare, per cui nessuno dei due può essere istanziato prima dell'altro.

Soluzione. La View comunica con il Controller attraverso l'interfaccia `GameViewListener`, fatta di callback che trasportano solo indici primitivi e mai oggetti del model. `GameViewImpl` mantiene un riferimento a un `GameViewListener` e lo propaga ai sottopannelli (`ConfigScreenView`, `PeekInitialCardsView`, `GameScreenView`), che traducono i click in chiamate ai callback. L'interfaccia Controller estende `GameViewListener`, quindi `ControllerImpl`, che implementa Controller, ne riceve i callback e reagisce agli eventi. Dipendendo solo da questa interfaccia, e non dalla classe concreta del Controller, la View resta disaccoppiata da esso: sostituirla richiede soltanto di reinvocare gli stessi callback. `ControllerImpl` riceve così gli input dell'utente tramite `GameViewListener`, oltre agli eventi del model tramite `GameObserver`. La dipendenza circolare tra i due è risolta con un'inizializzazione in due fasi: in `Main.start()` si costruisce prima la View, che a quel punto

non ha ancora un listener, poi il Controller, che riceve la View già costruita, e infine si chiama `view.setActionListener(controller)` per completare il collegamento inverso.

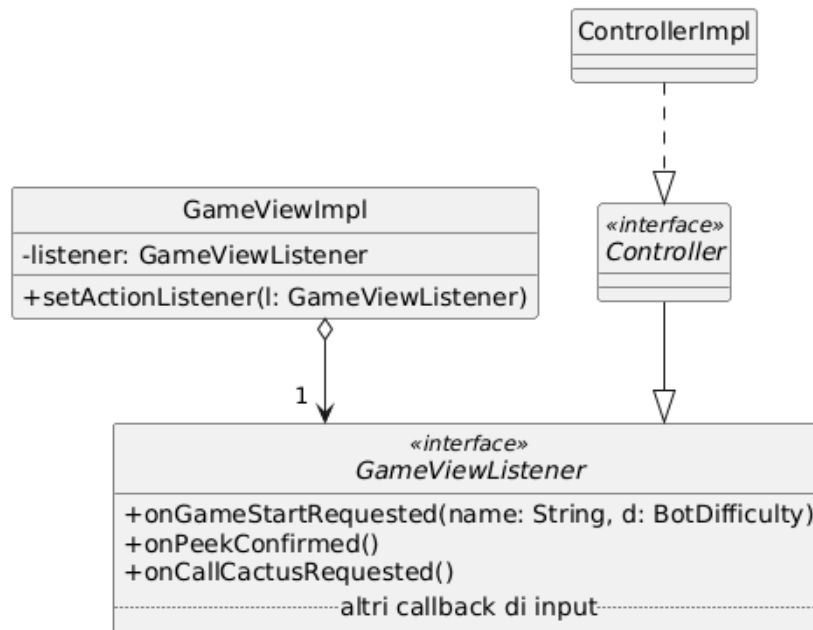


Figura 2.7: Rappresentazione UML della comunicazione dalla View verso il Controller tramite l'interfaccia `GameViewListener`.

Trasferimento dei dati verso la View

Problema. A ogni aggiornamento il Controller deve passare alla View uno snapshot completo e coerente dello stato di gioco utile alla visualizzazione: azioni disponibili, carta in cima agli scarti, mani dei giocatori, carta pescata e così via. Passare alla View il model vivo violerebbe la separazione MVC ed esporrebbe la View a uno stato mutabile, che potrebbe cambiare mentre viene letto; passare molti parametri sciolti sarebbe invece fragile e poco leggibile.

Soluzione. Si è introdotto `GameUpdateData`, un record che funge da Data Transfer Object immutabile dal Controller alla View. Essendo un record garantisce l'immutabilità di default (campi `final`, accessor generati, `equals`, `hashCode` e `toString`); il costruttore canonico compatto applica `List.copyOf` alle liste contenute, ottenendo copie difensive che rendono immutabili anche le collezioni, mentre due componenti sono di tipo `Optional` per modellare l'assenza senza ricorrere a `null`. Un caso particolare ha riguardato l'evidenziazione delle due carte scambiate quando un bot usa il potere `Swap`: riusare `SwapTarget`, che è un tipo del model, avrebbe creato una dipendenza dalla View verso il model. Si è quindi definito `SwapHighlight`, un piccolo record locale al package della View che trasporta solo i quattro indici interi; è il Controller a costruirlo a partire da `SwapTarget`, così il model resta isolato.

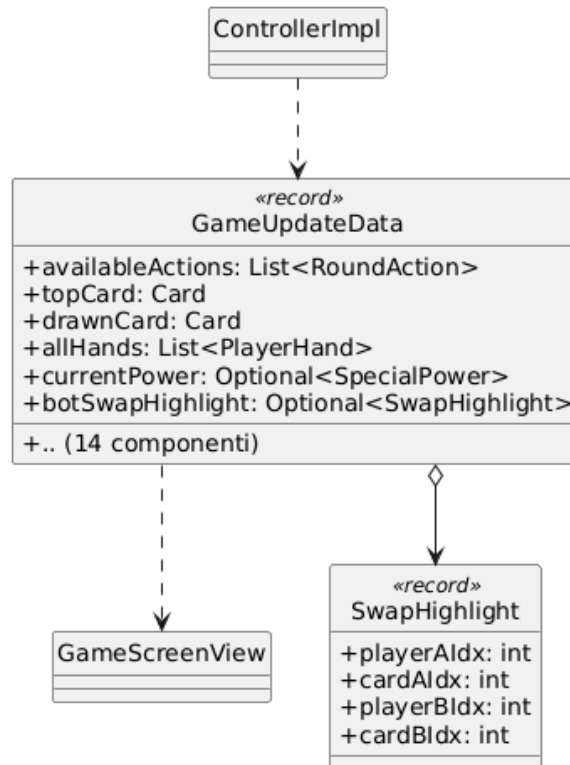


Figura 2.8: Rappresentazione UML del record immutabile GameUpdateData come DTO dal Controller verso la View.

2.2.3 Mondardini Arianna

Gestione Dinamica poteri speciali

Problema. Nel gioco Cactus, specifiche carte possiedono abilità speciali che si attivano durante il turno. Il problema architetturale consiste nel definire dove implementare la logica di questi poteri senza creare classi eccessivamente accentrate e difficili da estendere. Inoltre, poteri diversi richiedono parametri di input eterogenei (ad esempio, Scambiare richiede due giocatori e due indici, mentre Spiare richiede solo un giocatore e un indice), rendendo difficile definire una firma univoca per l'attivazione.

Soluzione. Per evitare di creare un codice rigido e con troppe classi basate sull'ereditarietà, abbiamo scelto la composizione applicando il pattern Strategy. Abbiamo isolato i poteri speciali nell'interfaccia SpecialPower, implementata da classi specifiche (PeekPower, RevealPower, SwapPower), lasciando alla carta il solo compito di richiamarli. Questa divisione rispetta il principio Open/Closed, per aggiungere un nuovo potere basterà creare una nuova classe, senza dover modificare il codice già scritto. Inoltre, per mantenere l'interfaccia pulita usando un solo metodo activate(), abbiamo applicato il refactoring Introduce Parameter Object. Invece di avere metodi con parametri tutti diversi, abbiamo racchiuso i dati necessari all'attivazione in un singolo parametro generale chiamato PowerTarget, definendo poi i dettagli specifici di ogni mossa tramite i Record di Java.

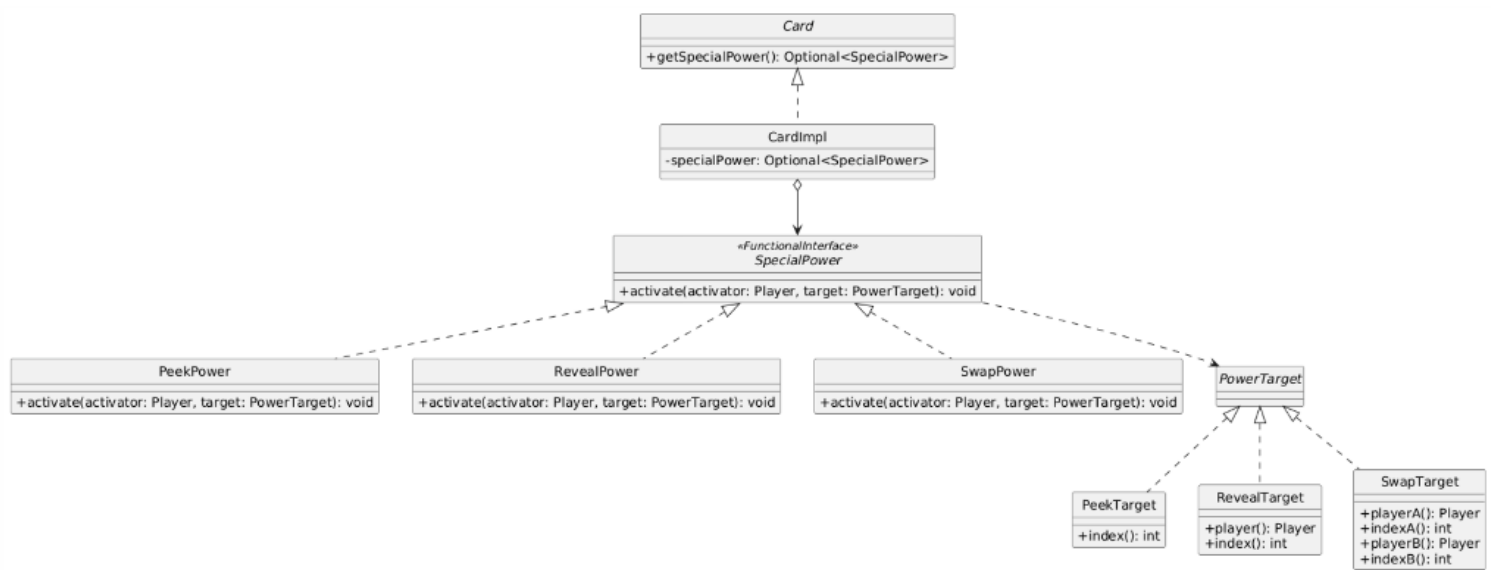


Figura 2.9: Rappresentazione UML dell' applicazione dei pattern Strategy e Parameter Object

Generazione del Mazzo e Regole di Dominio

Problema. La fase di setup del gioco richiede la creazione di un mazzo di 40 carte. Durante questa fase, è necessario applicare regole di dominio specifiche (ad esempio, assegnare il punteggio "0" alla carta di valore "10" e associare i poteri speciali alle carte 6, 7 e 8). Inserire le regole del gioco all'interno della classe che rappresenta la singola carta (CardImpl) avrebbe violato il Single Responsibility Principle.

Soluzione. Si è deciso di separare nettamente la rappresentazione della carta dalla logica della sua creazione. La classe CardImpl è stata modellata come un Immutable Object diventando un oggetto che si limita a contenere i dati della carta. Tutta la "conoscenza" del dominio e dell'assemblaggio è stata delegata alla classe DeckFactory applicando il pattern Static Factory Method tramite il metodo createBaseDeck(). Per evitare la creazione di oggetti DeckFactory non necessari, la classe è stata strutturata come Utility Class (dichiarata final e con costruttore privato). Così facendo, il controllore delega l'intera logica di creazione alla factory, ricevendo semplicemente un mazzo già assemblato e validato.

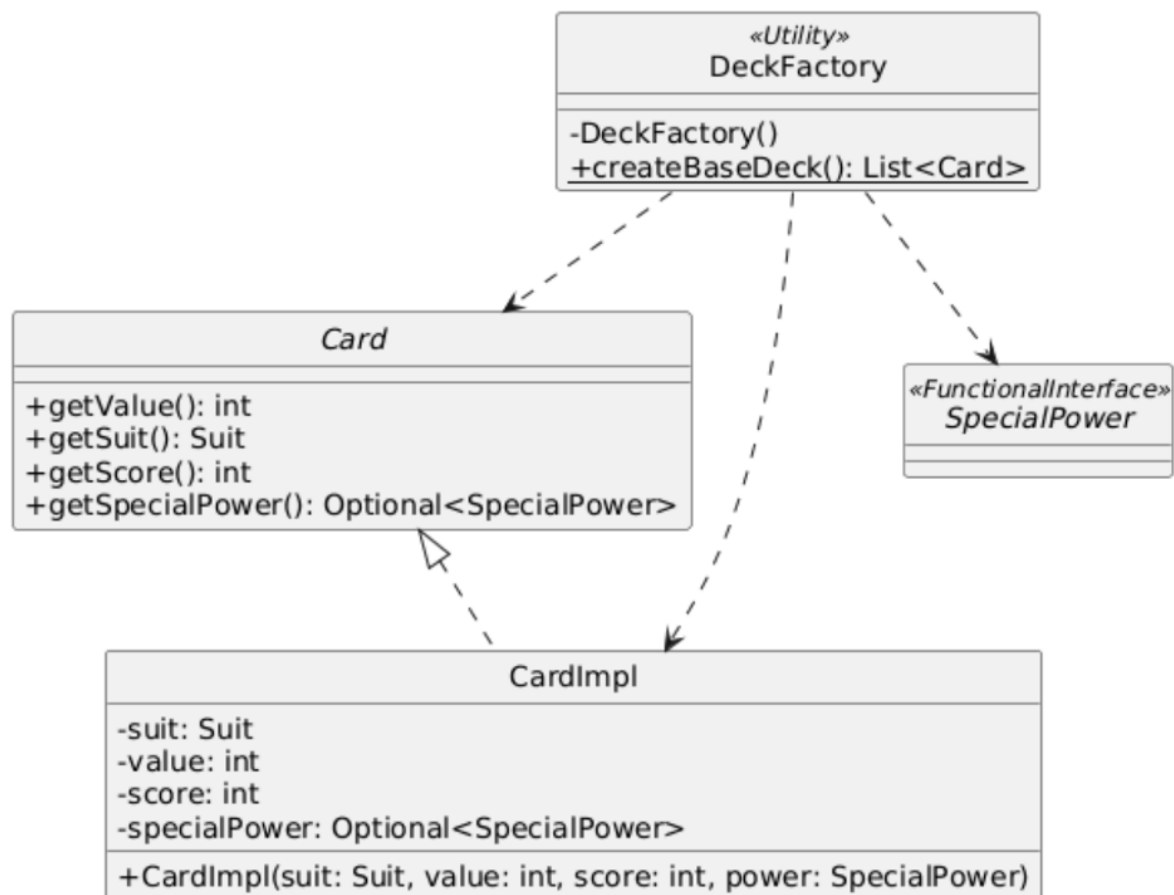


Figura 2.10: Applicazione del pattern Static Factory Method tramite la classe di utilità DeckFactory per la generazione validata del mazzo di gioco

Lo Stato Visivo e la Mano del Giocatore

Problema. Nel gioco, le carte posizionate davanti a un giocatore possono essere coperte o scoperte. Inizialmente si era ipotizzato di gestire questa caratteristica aggiungendo un attributo booleano `hidden` e i relativi metodi `setHidden()` direttamente all'interno della classe `CardImpl`. Tuttavia, lo stato "scoperta/coperta" non è una proprietà intrinseca dell'entità matematica "Carta" (il 7 di spade è tale indipendentemente da come è girato), ma è un'informazione contestuale legata allo stato della partita sul tavolo.

Soluzione. Mantenendo la carta invariabile, la responsabilità del suo orientamento è stata spostata nell'entità che la contiene (la mano del giocatore). Per farlo in modo coeso, all'interno di `PlayerHandImpl` è stata creata la classe interna privata `Slot`. Questa struttura applica il pattern Wrapper, "avvolgendo" l'oggetto `Card` originario per aggiungervi l'informazione sulla visibilità (boolean `hidden`), senza alterare la classe base. Inoltre, dichiarando privata la collezione di `Slot`, si applica rigorosamente l'Information Hiding in cui le interazioni esterne avvengono solo tramite i metodi dell'interfaccia `PlayerHand` (es. `revealCard` o `addCard`), garantendo che le regole strutturali del gioco non vengano violate (come l'impossibilità di superare il limite strutturale di sei carte).

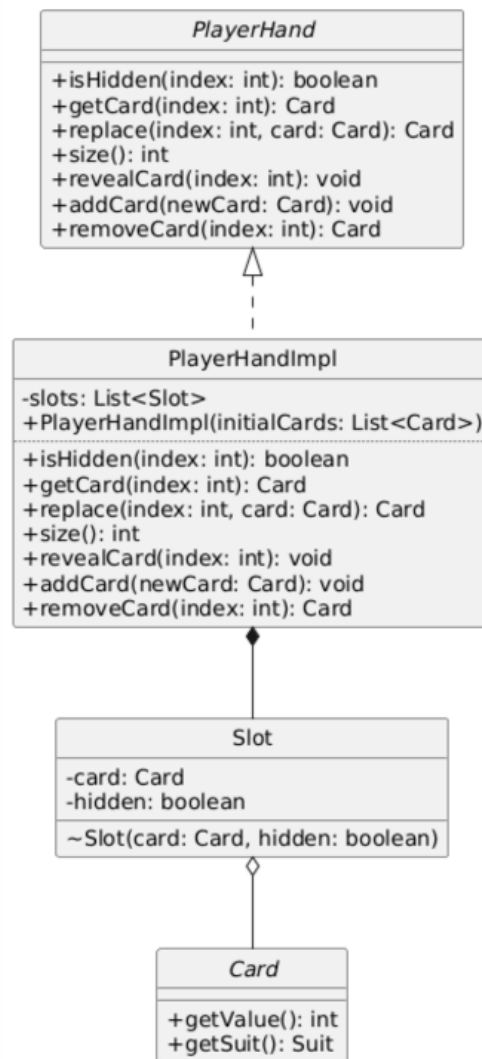


Figura 2.11: Struttura interna di PlayerHandImpl che utilizza la classe Slot come wrapper per mappare lo stato di visibilità delle carte.

2.2.4 Olivieri Rebecca

Azioni del turno

Problema. Ogni turno, sia del giocatore umano che dei bot, è composto da azioni distinte: pescare una carta, scartarla, scambiarla con una in mano, chiamare Cactus, attivare o saltare un potere speciale, tentare lo scarto simultaneo. Inserire la logica di ciascuna di queste azioni direttamente in RoundImpl l'avrebbe trasformata in una classe con troppe responsabilità, difficile da leggere, da testare e da estendere. Qualunque modifica a una singola azione avrebbe richiesto di riaprire il nucleo del model, con alto rischio di introdurre bug nelle parti già funzionanti.

Soluzione. Si è adottato il pattern Command. L'interfaccia RoundAction dichiara un unico metodo execute(MutableRound round) ed è annotata @FunctionalInterface. Ogni azione concreta (DrawAction, DiscardAction, SwapAction, CallCactusAction, EndTurnAction, ActivatePowerAction, SkipPowerAction, SimultaneousDiscardAction) è una classe o record separata che incapsula interamente la propria logica. RoundImpl si limita

a chiamare `action.execute(this)`, delegando l'esecuzione all'azione stessa senza conoscerne il tipo specifico. Per aggiungere una nuova azione o un nuovo potere speciale è sufficiente implementare una nuova classe `RoundAction`, senza modificare `RoundImpl` né nessun'altra parte del model, rispettando così il principio Open-Closed. L'alternativa di gestire tutte le azioni con un grande switch all'interno di `RoundImpl` avrebbe centralizzato tutta la logica in un unico punto, rendendo ogni aggiunta o modifica un intervento rischioso sul model.

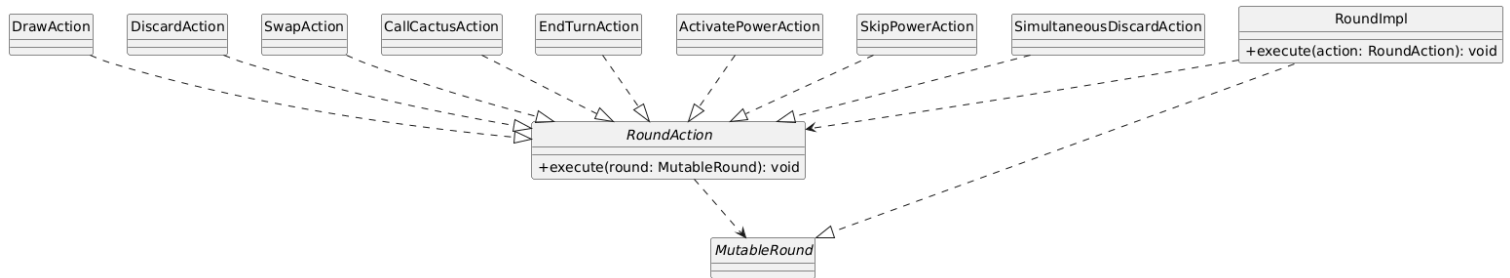


Figura 2.12: Rappresentazione UML del pattern Command applicato alle azioni del turno. Sono mostrate solo le relazioni rilevanti ai fini del pattern.

Fasi del turno

Problema. Le regole di Cactus impongono che il turno non sia lineare: a seconda della carta pescata o dell'azione scelta, alcune fasi possono essere saltate o aggiunte. La fase `SPECIAL_POWER` esiste solo se la carta scartata possiede un potere speciale, altrimenti viene saltata completamente. Le azioni disponibili cambiano completamente a seconda del momento in cui ci si trova nel turno. Gestire questa logica con variabili booleane o catene di if-else avrebbe prodotto un codice fragile e difficile da estendere con nuove fasi.

Soluzione. Si è implementata una macchina a stati finiti all'interno di `RoundImpl` tramite l'enumerazione `TurnPhase`, applicando il pattern State. Gli stati del turno sono: `DRAW`, `DECISION`, `SPECIAL_POWER` (opzionale), `END_TURN`, `SIMULTANEOUS_DISCARD`, `ENDED`. Tutta la logica di transizione è centralizzata nel metodo `advancePhase()`: la transizione più significativa è quella da `DECISION`, dove il prossimo stato dipende dal contenuto della carta pescata, se ha un potere speciale si va in `SPECIAL_POWER`, altrimenti si salta direttamente a `END_TURN`. Il metodo `getAvailableActions()` usa uno switch expression sulla fase corrente per restituire alla view esclusivamente le azioni valide in quel momento: la view riceve sempre una lista già filtrata, senza dover conoscere la fase corrente, mantenendo netta la separazione tra logica di controllo e presentazione.

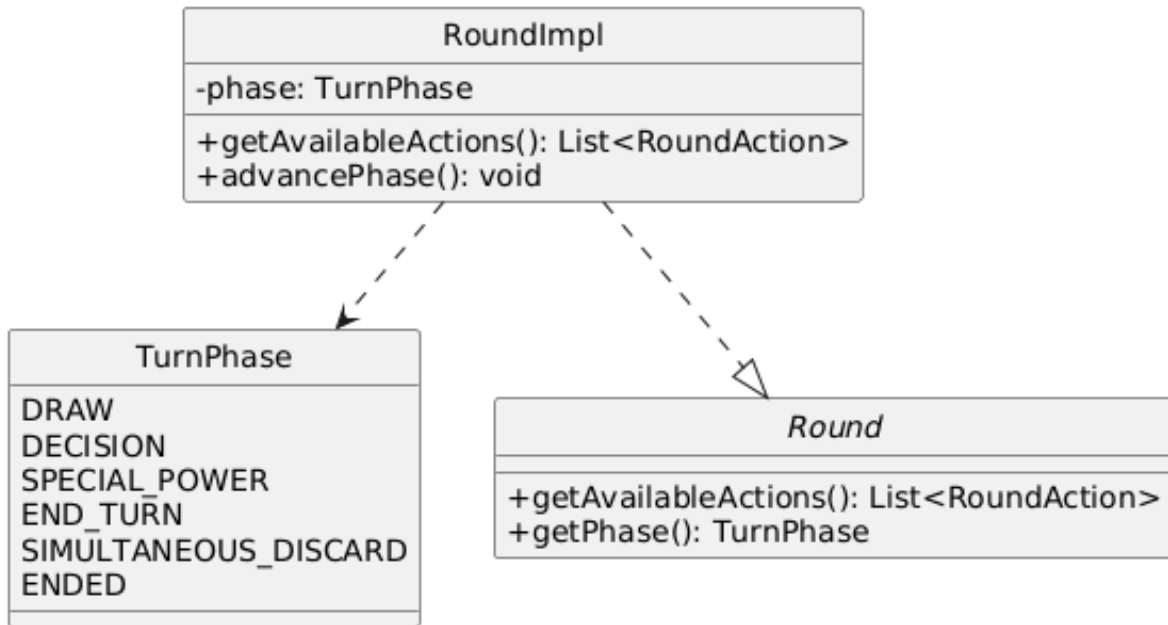


Figura 2.13: Rappresentazione UML del pattern State applicato alle fasi del turno.

Notifica degli eventi di gioco

Problema. Il model deve comunicare al controller quando lo stato della partita cambia in modo significativo: avanzamento al turno successivo, modifica dello stato durante il turno corrente, fine della partita. Richiamare direttamente il controller da GameImpl accoppierebbe il model alla sua logica di controllo, violando la separazione propria di MVC e rendendo impossibile riutilizzare o testare il model indipendentemente.

Soluzione. Si è adottato il pattern Observer. In questo pattern GameImpl ricopre il ruolo di observable, in quanto è la sorgente degli eventi di gioco, mentre il Controller ricopre il ruolo di observer. L'interfaccia GameObserver dichiara tre metodi di notifica: onRoundAdvanced(), onGameFinished() e onGameStateChanged(). GameImpl mantiene internamente una lista di observer e li notifica nei momenti opportuni. Il controller si registra come observer all'avvio tramite addObserver e reagisce agli eventi aggiornando la view, senza che GameImpl lo conosca o lo importi. L'aggiunta di un nuovo tipo di evento richiede solo un nuovo metodo in GameObserver e la relativa notifica in GameImpl. In questo modo future modifiche al controller, o addirittura la sua completa sostituzione, non richiederebbero alcuna modifica al model.

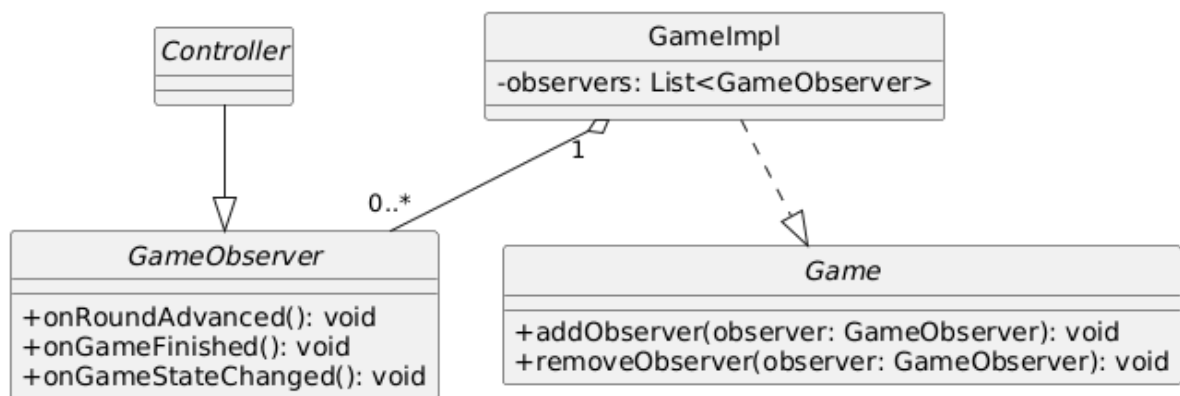


Figura 2.14: Rappresentazione UML del pattern Observer applicato alla notifica degli eventi di gioco.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Tutti i test sono stati realizzati con JUnit ed eseguiti tramite il build system Gradle. I test coprono le componenti principali del model, dalle quali dipendono tutte le altre funzionalità del sistema, al fine di attestare l'affidabilità del codice e preservarne l'integrità durante l'evoluzione del progetto.

Test riguardanti le carte e i mazzi

- **CardTest:** viene verificata la corretta inizializzazione delle carte, le regole specifiche del gioco (ad esempio il Re con valore 10 ha punteggio 0) e la robustezza del costruttore rispetto a parametri non validi.
- **DiscardPileTest:** viene verificato il corretto comportamento del mazzo degli scarti, inclusi lo scarto di carte, il controllo della carta in cima e lo svuotamento del mazzo.
- **DrawPileTest:** viene verificato il comportamento del mazzo di pesca, inclusi la pesca delle carte, il controllo dello stato vuoto e il riempimento del mazzo.

Test riguardanti mani e giocatori

- **PlayerTest:** viene verificata la corretta inizializzazione di `HumanPlayer` e `BotPlayer`, inclusa la gestione degli errori per operazioni non valide come ottenere la mano prima che essa venga assegnata.
- **PlayerHandTest:** viene verificato il comportamento della mano del giocatore, inclusi l'inizializzazione, la rivelazione di carte, la sostituzione di carte e la gestione degli indici non validi.

Test riguardanti la logica di gioco

- **RoundTest:** viene verificato il corretto avanzamento delle fasi del turno e il comportamento di tutte le azioni disponibili.
- **GameTest:** viene verificata l'inizializzazione della partita, la rotazione dei giocatori, il conteggio dei round completati e la gestione degli errori per operazioni non valide.

- **SpecialPowerTest:** viene verificata la logica dei tre poteri speciali (PeekPower, RevealPower e SwapPower), includendo anche la gestione di target non validi tramite eccezioni.

Test riguardanti punteggio e statistiche

- **StatsTest:** viene verificato il calcolo delle statistiche di gioco, inclusi conteggio vittorie, classifica generale e media dei round completati.
- **ScoreTest:** viene verificato il calcolo dei punteggi e la determinazione del vincitore tramite ScoreCalculator e GameResult.

Test del controller

- **ControllerTest:** verifica il corretto funzionamento del controller, testando l'avvio della partita, la gestione delle azioni e il salvataggio dei risultati a fine partita tramite una view e un repository fittizi.

Test riguardanti le strategie dei bot

- **EasyBotStrategyTest, MediumBotStrategyTest:** viene verificato che le strategie senza memoria scelgano un'azione valida in ogni fase del turno e rispettino la propria logica di base.
- **HardBotStrategyTest, VeryHardBotStrategyTest:** viene verificato il comportamento delle strategie con memoria, inclusi la memorizzazione delle carte spiate, le decisioni di scambio e scarto guidate dalla memoria e la chiamata di "Cactus!" solo nelle condizioni previste.

Infine, l'interfaccia grafica è stata testata manualmente durante lo sviluppo del software.

3.2 Note di sviluppo

Mazzoni Chiara

Utilizzo di Stream e lambda expressions: Usate spesso nel progetto, un esempio è il calcolo della classifica generale.

<https://github.com/Rebe01i/00P25-Cactus/blob/f3a841922e5e97a0e43b4c24507ce378b6e9c052/src/main/java/it/unibo/cactus/model/statistics/StatsCalculatorImpl.java#L22-L29>

Uso di Optional: Utilizzati per la gestione sicura della pesca delle carte dalle pile di gioco, evitando la restituzione di riferimenti null.

<https://github.com/Rebe01i/00P25-Cactus/blob/f3a841922e5e97a0e43b4c24507ce378b6e9c052/src/main/java/it/unibo/cactus/model/pile/DrawPileImpl.java#L33-L36>

Utilizzo di Stats della libreria Google Guava (com.google.common.math.Stats): Utilizzata per il calcolo efficiente e robusto della media matematica dei round giocati.

<https://github.com/Rebe01i/00P25-Cactus/blob/7a7c46700c4a3270ca82c9ea6dd782529e1fe1f0/src/main/java/it/unibo/cactus/model/statistics/StatsCalculatorImpl.java#L31-L41>

Utilizzo della libreria Gson (com.google.gson.Gson): Utilizzata per la serializzazione e deserializzazione dei risultati di gioco in formato JSON.

<https://github.com/Rebe01i/00P25-Cactus/blob/7d4d3327db86e457e548d9ed05a7fc3c571b7094/src/main/java/it/unibo/cactus/model/statistics/JSonHistoryRepository.java#L35-L46>

Utilizzo di Record: Utilizzati per garantire l'immutabilità dei dati nella restituzione del risultato della partita e nel fornire i dati per le statistiche.

<https://github.com/Rebe01i/00P25-Cactus/blob/7d4d3327db86e457e548d9ed05a7fc3c571b7094/src/main/java/it/unibo/cactus/model/score/GameResult.java#L13-L22>

Utilizzo di JavaFX: Impiegato per lo sviluppo delle classi di View.

<https://github.com/Rebe01i/00P25-Cactus/blob/7d4d3327db86e457e548d9ed05a7fc3c571b7094/src/main/java/it/unibo/cactus/view/DiscardPileView.java#L1>

Molari Sofia

Utilizzo di Stream e lambda expressions: Impiegati nelle strategie dei bot, ad esempio per sommare il punteggio delle carte note in memoria.

<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/model/players/strategies/SelfBotMemory.java#L46>

Utilizzo di Optional: Usato per gestire in modo sicuro la carta pescata in fase di decisione, evitando riferimenti null.

<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/model/players/strategies/AbstractMemoryBotStrategy.java#L81>

Utilizzo di switch expression: Impiegata in AbstractBotStrategy per selezionare il passo da eseguire in base alla fase corrente del turno.

<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/model/players/strategies/AbstractBotStrategy.java#L18-L25>

Utilizzo di Record: Usato per il DTO immutabile GameUpdateData, con costruttore canonico compatto e copie difensive tramite List.copyOf.

<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/view/GameUpdateData.java#L35-L58>

Utilizzo di Collections.unmodifiableMap: Usato per esporre la memoria del bot come vista non modificabile su una copia della mappa interna.

<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/model/players/strategies/SelfBotMemory.java#L52>

Utilizzo di JavaFX: Impiegato nelle schermate di mia competenza, ad esempio per il binding dell'altezza delle carte tramite `Bindings.createDoubleBinding` in `PeekInitialCardsView`.
<https://github.com/Rebe01i/00P25-Cactus/blob/7c00c4f66406e77cfe86e9d9d3eb678f04979f76/src/main/java/it/unibo/cactus/view/PeekInitialCardsView.java#L58>

Mondardini Arianna

Utilizzo di Optional

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/c9cb16313fa44e38ba86f8760e9ba2bade3912eb/src/main/java/it/unibo/cactus/model/cards/CardImpl.java#L13>

Creazione e utilizzo di Interfacce Funzionali

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/c9cb16313fa44e38ba86f8760e9ba2bade3912eb/src/main/java/it/unibo/cactus/model/cards/SpecialPower.java>

Utilizzo di Stream e lambda expressions

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/d4845249874eb83cfa253908b398792d908acd8/src/main/java/it/unibo/cactus/model/players/PlayerHandImpl.java#L26-L33>

Utilizzo di Record

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/c9cb16313fa44e38ba86f8760e9ba2bade3912eb/src/main/java/it/unibo/cactus/model/cards/target/PeekTarget.java#L3>

Utilizzo di javaFX

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/d4845249874eb83cfa253908b398792d908acd8/src/main/java/it/unibo/cactus/view/TableView.java>

Olivieri Rebecca

Utilizzo di Stream e lambda expressions

Usate in più punti del progetto, un esempio è il seguente.

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/f3a841922e5e97a0e43b4c24507ce378b6e9c052/src/main/java/it/unibo/cactus/model/game/GameImpl.java#L100-L105>

Utilizzo di Optional

Utilizzati in vari punti del progetto. Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/0c25570c1581fd9448e23a5a11055adc2418adeb/src/main/java/it/unibo/cactus/model/rounds/RoundImpl.java#L144-L147>

Utilizzo di Record

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/0c25570c1581fd9448e23a5a11055adc2418adeb/src/main/java/it/unibo/cactus/model/rounds/actions/SwapAction.java#L16>

Utilizzo di JavaFX

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/9e209f074b1effc4aa2275a91534d2cec3280be4/src/main/java/it/unibo/cactus/view/SimultaneousDiscardOverlay.java>

Utilizzo di Interfacce Funzionali

Permalink: <https://github.com/Rebe01i/00P25-Cactus/blob/0c25570c1581fd9448e23a5a11055adc2418adeb/src/main/java/it/unibo/cactus/view/MenuOverlay.java#L25-L34>

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Mazzoni Chiara

Personalmente mi ritengo molto soddisfatta del risultato finale dell'applicazione e del mio codice. Essendo questo il primo progetto software a cui abbia preso parte, le idee iniziali erano inevitabilmente un po' confuse. Tuttavia, credo che l'approccio di tipo top-down che abbiamo seguito e l'attenzione che abbiamo dedicato alla strutturazione dell'architettura nei primi giorni, siano stati fondamentali per raggiungere l'attuale risultato.

Sono molto contenta della collaborazione che si è creata nel gruppo. In particolare, credo che la suddivisione mirata delle parti abbia permesso un'implementazione fluida e pratica, permettendoci di evitare grossi intoppi e di integrare bene il codice di tutti.

All'interno di questa organizzazione, il mio contributo è stato trasversale ai tre livelli dell'architettura MVC, in quanto mi sono occupata della progettazione e dello sviluppo della parte di Model riguardante i mazzi, i punteggi e le statistiche, e di conseguenza ho implementato le corrispondenti interfacce grafiche nella parte di View. Infine, ho contribuito attivamente nello sviluppo del Controller.

A livello personale e professionale, questo progetto è stato un'importante occasione di crescita. Mi ha dato l'opportunità di imparare molto e di utilizzare strumenti di sviluppo essenziali che prima non conoscevo, come le librerie Google Guava e Gson. Inoltre, l'utilizzo di Git si è presto rivelato indispensabile per coordinare la gestione del codice.

Tra i possibili sviluppi futuri credo sarebbe interessante espandere il calcolatore delle statistiche, inserendo un sistema di obiettivi sbloccabili e grafici, che permettano di monitorare l'andamento delle vittorie dei giocatori nel tempo.

4.1.2 Molari Sofia

Affronto questo progetto da sviluppatrice C++ da alcuni anni, una posizione un po' diversa dal resto del gruppo. La differenza che ho sentito di più è stata il cambio di paradigma rispetto al C++, dal tradurre costrutti che do per scontati al rinunciare alla gestione manuale della memoria. A questo si è aggiunta una differenza di metodo: sul lavoro si segue un approccio più flessibile e meno rigoroso su pratiche come la documentazione o i test automatici, e tornare al rigore accademico me ne ha fatto riapprezzare il valore, soprattutto sul testing.

All'interno del gruppo mi sono occupata del modello dei giocatori e dell'intera gerarchia

delle strategie dei bot, oltre all'infrastruttura di comunicazione tra View e Controller (l'interfaccia `GameEventListener`, il record `GameUpdateData` e le schermate di configurazione e di peek iniziale). Avendo già esperienza con Qt, il paradigma di JavaFX non mi era nuovo; l'esperienza lavorativa, inoltre, ha aiutato su aspetti pratici come le stime sui tempi di sviluppo e il bugfixing. Le mie compagne sono state autonome e solide, ognuna padrona della propria parte.

Il punto di forza del mio lavoro è la gerarchia delle strategie dei bot, estensibile e con la logica comune ben fattorizzata. Una conferma è arrivata quando, provando il gioco, ci siamo accorte che i tre livelli iniziali erano poco distinti e poco sfidanti: ho aggiunto un quarto livello a sviluppo già avviato e la struttura lo ha assorbito senza intaccare le strategie esistenti. Come punto di debolezza riconosco che l'astrazione `BotMemory` ha oggi un solo implementatore, `SelfBotMemory`, e risulta quindi un po' superflua: l'ho mantenuta perché è semplice e lascia aperta la porta a memorie diverse.

In futuro l'estensione più sensata sarebbe un bot capace di contare le carte, ricordando quelle già passate dagli scarti per stimare cosa resta nel mazzo: è l'unico caso che giustificherebbe davvero una memoria più ricca, mentre estenderla al tavolo o agli altri giocatori non avrebbe senso con le regole attuali.

4.1.3 Mondardini Arianna

All'interno del team il mio contributo principale ha riguardato la progettazione della logica di dominio delle carte e della mano di gioco nel Model. Sulla base di questo lavoro, ho successivamente sviluppato il Controller e realizzato l'interfaccia grafica in JavaFX, occupandomi dell'integrazione tra la logica applicativa e la componente visiva secondo il pattern MVC. La parte del progetto che ho trovato più stimolante è stata l'intero percorso che porta dalla definizione della logica di gioco fino alla sua rappresentazione grafica, trasformando strutture dati e regole astratte in elementi visivi con cui l'utente può interagire. È stata la mia prima esperienza di lavoro di gruppo su un progetto di tale portata e, oltre all'esito positivo della collaborazione, ritengo che il prodotto finale ripaghi pienamente tutto l'impegno e la dedizione che vi abbiamo investito. Il supporto frequente che ci siamo date a vicenda per tutta la stesura del progetto ha reso le ore di lavoro decisamente più leggere. Inoltre, mi sono occupata personalmente di disegnare da zero tutte le carte del mazzo e di ideare l'interfaccia grafica del gioco, partendo da schizzi su carta fino ad arrivare alla realizzazione finale in JavaFX, vivendo questo progetto anche come passatempo personale. Essendo alle prime armi, nelle fasi iniziali la curva di apprendimento di JavaFX ha portato alla stesura di un codice grafico che definirei ingenuo. Questo approccio iniziale ha richiesto successivi e importanti refactoring per evitare sovrapposizioni visive e risolvere problemi. Col senno di poi, una padronanza anticipata di questi concetti sarebbe stata d'aiuto. Per possibili lavori futuri, il progetto Cactus possiede un'architettura sufficientemente solida per supportare diverse espansioni. Una prima evoluzione sarebbe l'implementazione di una modalità multiplayer online, sostituendo i Bot attuali con un'architettura client-server. Dal punto di vista grafico, mi piacerebbe arricchire ulteriormente con animazioni 3D o con l'inserimento di ulteriori dettagli visivi e interattivi, in particolare reintegrando un cortometraggio animato che avevo già prodotto, ma che è stato escluso nella build finale per preservare la compatibilità multiplatforma dell'applicazione.

4.1.4 Olivieri Rebecca

Sono soddisfatta del risultato finale ottenuto, sia per quanto riguarda il progetto che il lavoro di squadra. All'interno del gruppo mi sono occupata principalmente della logica del turno di gioco, progettando il flusso delle fasi e le azioni disponibili per ciascuna di esse. Seguendo l'architettura MVC adottata, ho contribuito a tutte e tre le componenti: ho sviluppato il model relativo al flusso di gioco, le sue interfacce della view e ho partecipato allo sviluppo del controller. La fase più impegnativa è stata quella iniziale di analisi: capire come le varie parti si collegassero tra loro ha richiesto molto tempo e riflessione prima di passare al codice. Una volta chiarita la struttura, però, lo sviluppo è proceduto in modo fluido, e mi ha permesso di capire quanto una buona progettazione iniziale semplifichi tutto il lavoro successivo.

Il punto di forza del mio lavoro è stata proprio la cura nella progettazione iniziale: la struttura realizzata è solida ed estendibile. Aggiungere una nuova azione di gioco richiede semplicemente di implementare una nuova classe `RoundAction`, mentre aggiungere una nuova fase del turno richiede di estendere `TurnPhase` e aggiungere i relativi casi in `RoundImpl`, senza toccare il resto del model. Come punto di debolezza, riconosco l'accoppiamento tra `RoundImpl` e `Game`, necessario per gestire la fase di scarto simultaneo: `RoundImpl` accede direttamente alla lista dei giocatori tramite `game.getPlayers()`. Questo legame potrebbe essere ridotto, rendendo il codice più pulito.

Per quanto riguarda il lavoro di gruppo, abbiamo sempre utilizzato branch separati su Git, il che ha reso l'integrazione ordinata e ha evitato conflitti. Ciascuna ha rispettato le proprie parti, ma non è mancato l'aiuto reciproco. Considerando che si tratta della prima esperienza su un progetto di questa complessità, ritengo che il risultato ottenuto sia più che soddisfacente e che l'esperienza sia stata formativa, soprattutto in vista del mondo lavorativo.

In futuro la struttura del turno potrebbe essere estesa aggiungendo nuove azioni, nuove fasi di gioco o nuovi poteri speciali, grazie alle scelte di design adottate che rendono queste modifiche localizzate e non invasive.

Appendice A

Guida utente

Questo capitolo illustra le funzionalità principali dell'applicativo e fornisce le istruzioni necessarie per permettere a un nuovo utente di familiarizzare rapidamente con l'interfaccia e le meccaniche di gioco di Cactus.

A.1 Configurazione iniziale

All'avvio dell'applicazione, l'utente accede alla schermata iniziale di configurazione. Da qui può consultare le regole tramite il pulsante "Rules" e selezionare il livello di difficoltà. Infine, per poter avviare la partita, è richiesto l'inserimento obbligatorio del proprio nome nell'apposito campo di testo.

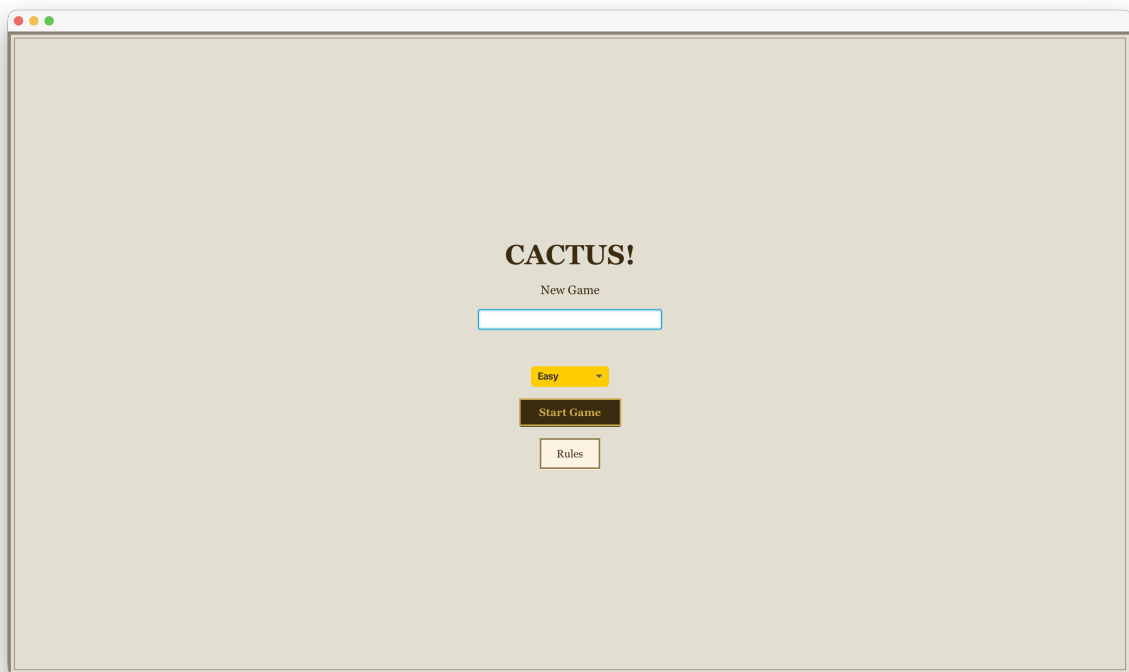


Figura A.1: Schermata di configurazione

A.2 Schermata di gioco

All'avvio del gioco selezionare due delle quattro carte disponibili, così da poterle vedere.

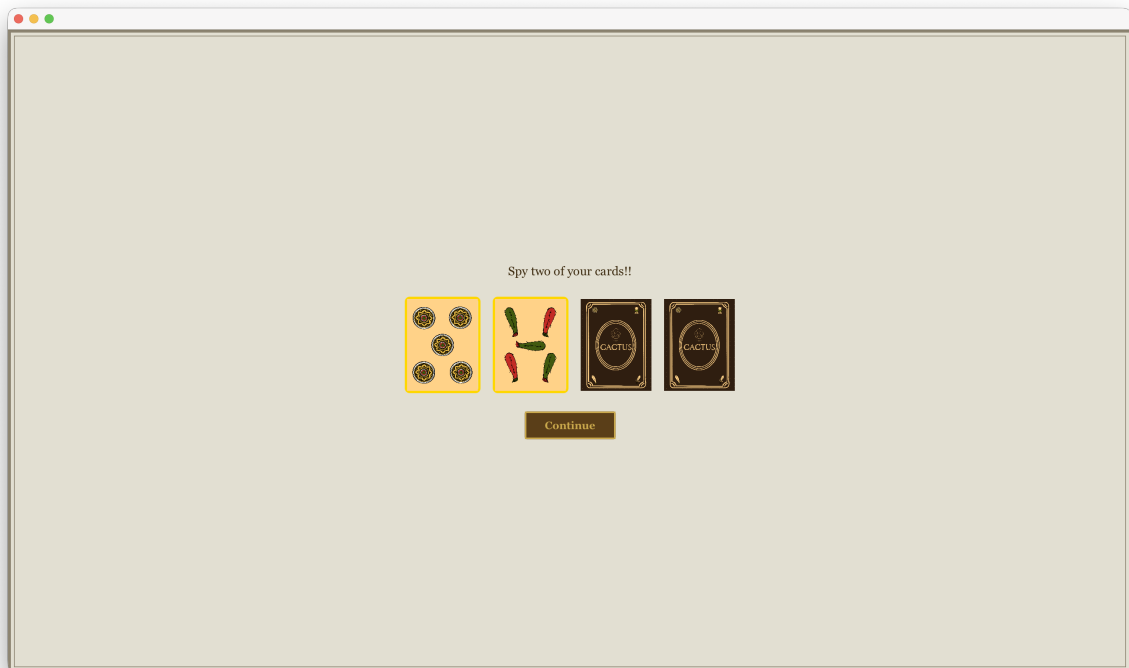


Figura A.2: Scelta di due carte da spiare

Successivamente l'utente accede alla schermata principale, dove gioca contro tre bot. L'interfaccia è divisa in tre aree: l'area di gioco con mazzi della pesca e degli scarti e mani dei giocatori; la barra superiore con informazioni sul turno e menu; e il pannello inferiore con azioni disponibili e suggerimenti per l'utente.

Durante il proprio turno è possibile:

- pescare una carta, selezionando il mazzo della pesca;
- scartare una carta, selezionando la carta che si desidera scartare;
- per le altre mosse possibili, seguire le istruzioni nella barra in basso.

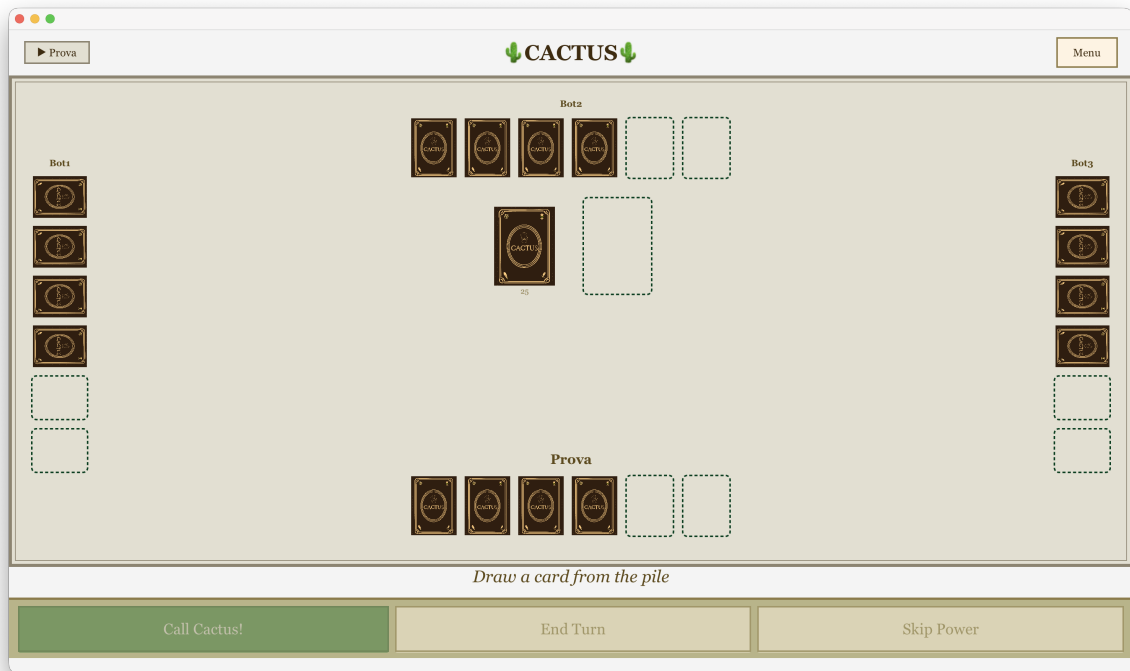


Figura A.3: Tavolo di gioco

A.3 Scarto simultaneo

Al termine del turno di un qualsiasi giocatore compare la schermata di scarto simultaneo, selezionare una carta per riuscire a scartarla prima degli altri giocatori.

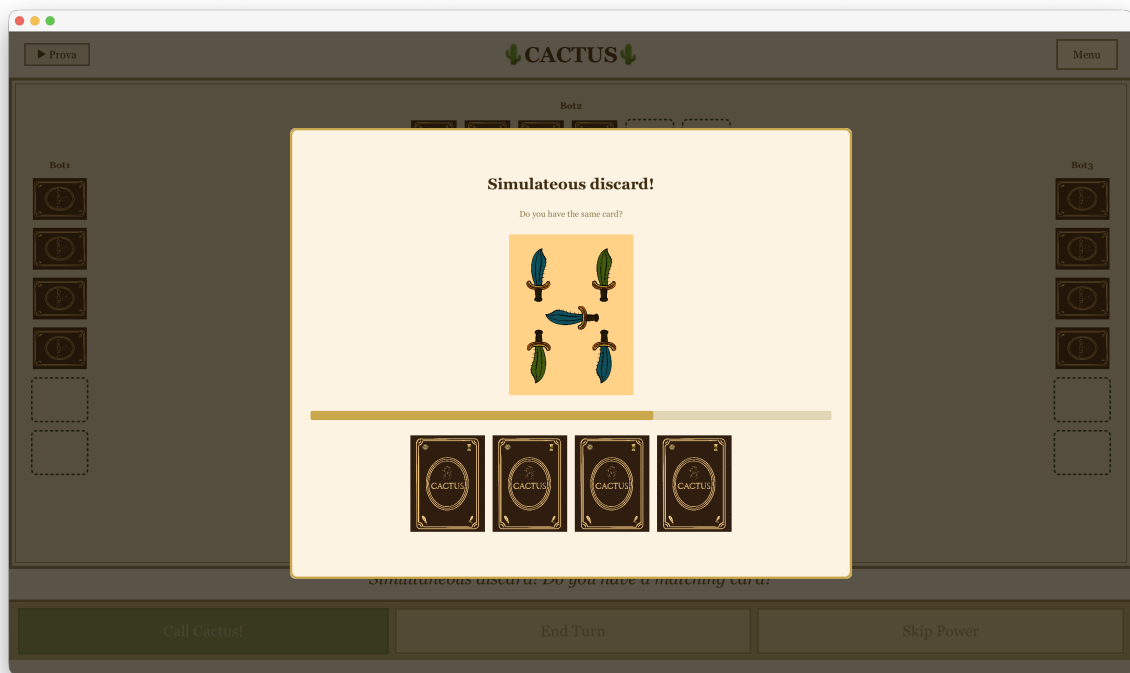


Figura A.4: Scarto simultaneo

A.4 Specifica delle carte del gioco

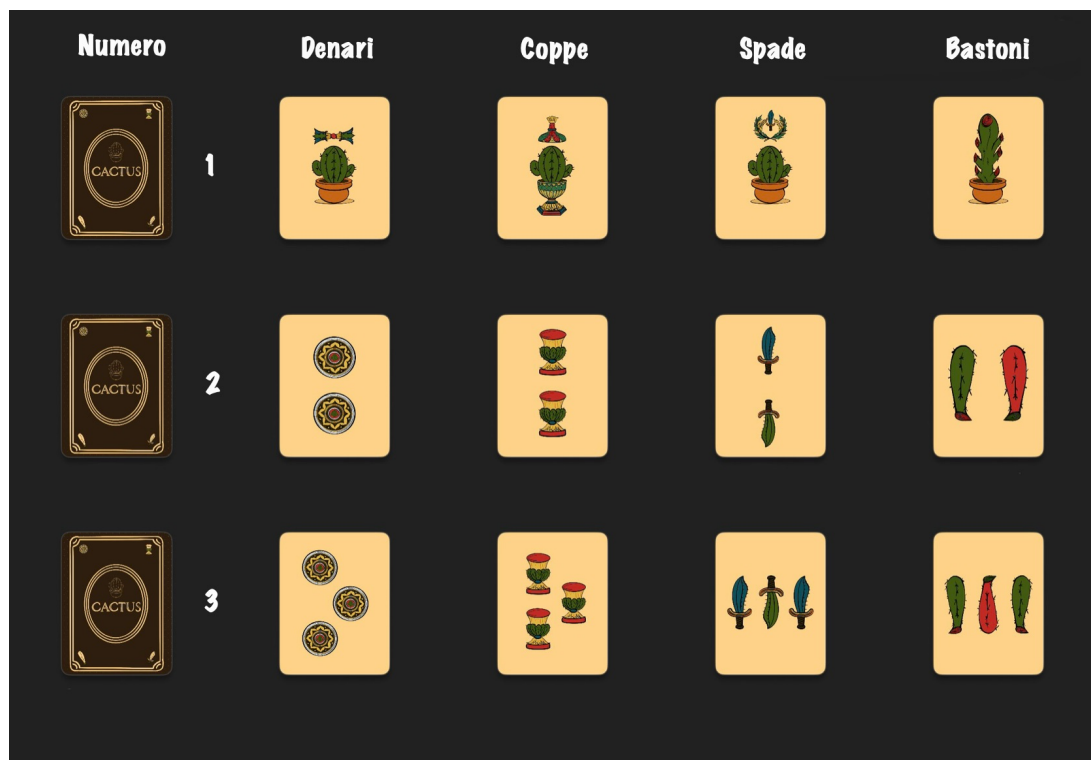


Figura A.5: Carte di gioco 1-3

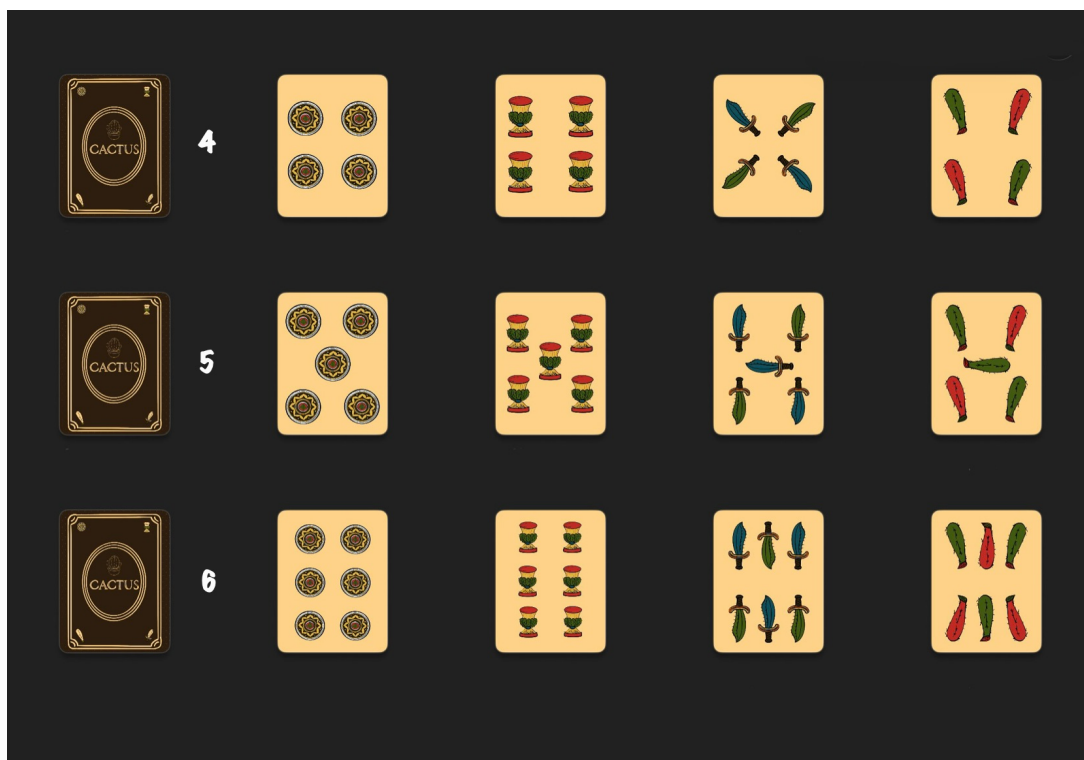


Figura A.6: Carte di gioco 4-6

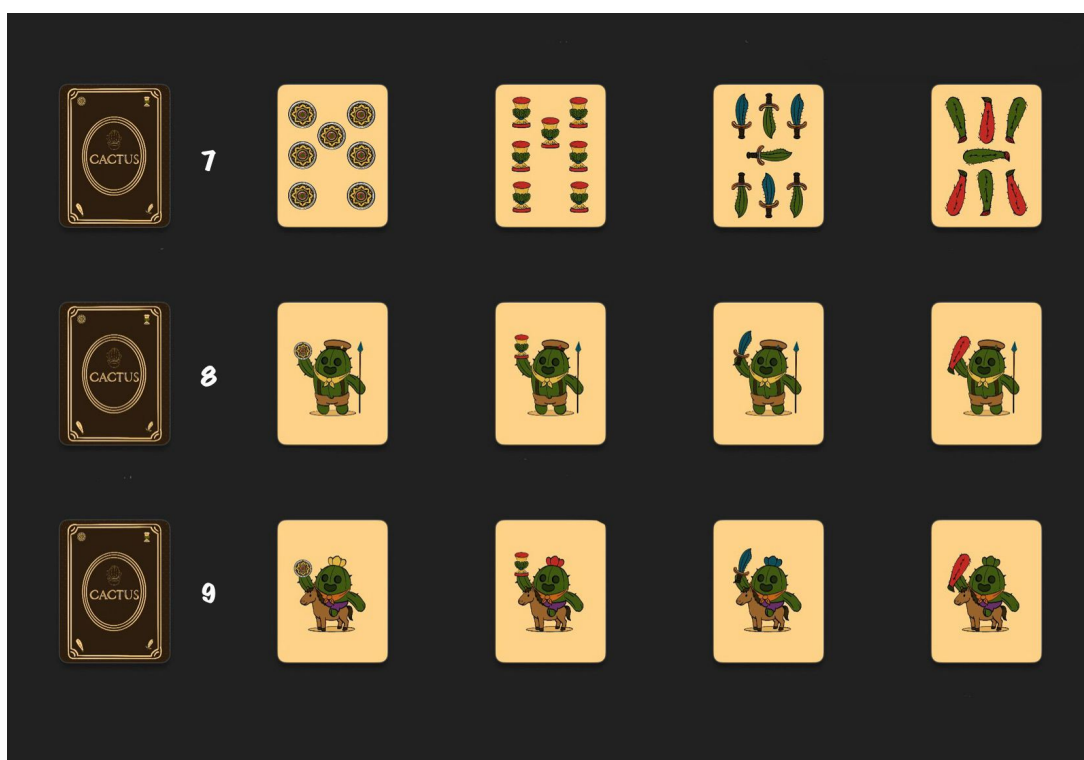


Figura A.7: Carte di gioco 7-9

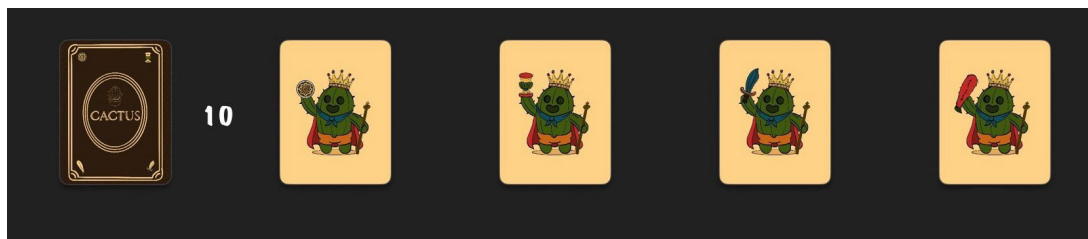


Figura A.8: Carta di gioco 10

Appendice B

Esercitazioni di laboratorio

B.0.1 chiara.mazzoni12@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284989>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286153>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287129>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288454>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289566>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290681>

B.0.2 sofia.molari@studio.unibo.it

B.0.3 arianna.mondardini@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284988>
- Laboratorio 08: <https://github.com/AriannaMonda/lab08>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287239>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288478>
- Laboratorio 11: <https://github.com/AriannaMonda/lab11>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290668>

B.0.4 rebecca.olivieri2@studio.unibo.it

- Laboratorio 07:<https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284990>
- Laboratorio 08:<https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286134>
- Laboratorio 09:<https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287130>
- Laboratorio 10:<https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288474>
- Laboratorio 11:<https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289565>
- Laboratorio 12:<https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290687>